doc-integrity

Revision: 1 **Last modified:** 2026-07-08T08:30:00Z

A project-agnostic (§11.4.28) Go tool that runs **five machine-checkable check families** over a consumer-registered doc-set and REFUSES export/commit on any integrity FAIL — the mandatory-before-export gate of constitution **§11.4.186** (design: `docs/research/doc_integrity_validator/DESIGN.md`).

It catches the class of doc defects that human eyeballing missed: duplicate rows for one feature with divergent timelines, `MVP↔V9-plan` cross-doc divergences, deadline-before-dependency, Status↔Type mismatches, and column/id-structure drift.

Check families

Family	Rules	What it asserts
DEDUP	`DEDUP-01/02`	No two rows describe the same functionality — keyed on `ticket` OR normalised `(subject, scope)` (NOT bare subject substring — that false-positives on distinct NanoKVM tasks, §11.4.6).
TIMELINE	`TIME-01/01b/02/03/04`	`start ≤ deadline`; no deadline-before-dependency; no dependency cycle; section span; `GATED` exempt-but-marked.
CROSS-DOC	`XDOC-01`	The same item across plan/MVP/summary/DB (joined by ticket, fuzzy-subject fallback) carries identical timeline/status/type; authoritative source named.
INTEGRITY	`INTEG-01/02/03`	No orphan refs; Status↔Type agreement (§11.4.33 Bug→Fixed / Feature→Implemented / Task→Completed); location↔status agreement.
STRUCTURAL	`STRUCT-01/02`	Required columns present; item IDs unique + match `id_pattern` (+ opt-in monotonic).

Adapters (source kinds)

`xlsx` (excelize) · `markdown-table` · `markdown-headings` (Issues/Fixed H2) · `sqlite` (READ-ONLY — the `workable_items.db`). New source kind = new adapter, never a project fork.

Usage

```
# Build
go build -o doc-integrity ./cmd/doc_integrity

# Validate a consumer doc-set (checkset first, flags after):
doc-integrity verify <checkset.yaml> --repo-root . [--json out.json] [--evidence-dir dir] [--quiet]
doc-integrity report <checkset.yaml> --repo-root .
doc-integrity selfcheck        # runs the embedded golden fixtures (§11.4.107(10))
doc-integrity version
```

Exit codes: `0` PASS · `1` FAIL (findings; export MUST be refused) · `2` config error · `3` SKIP (a source is unavailable — honest §11.4.3, never a fake PASS).

Every finding names the offending row/item (`sheet!A47` , `path:line` , `items.<id>`) so it is actionable (§11.4.6).

Consumer configuration (project data lives in YAML, never in Go)

The consumer registers its doc-set + rules in `.<consumer>/doc_integrity/checkset.yaml` (mirrors the `.docs_chain/contexts/*` precedent). Full schema + example: `DESIGN.md §1.4` .

Anti-bluff self-validation (§ . . ( ))

`selfcheck` runs embedded golden fixtures under `internal/selfcheck/golden/` : golden-good PASS, one golden-bad per check family FAIL (pinpointing the injected offender), and a **negative-control** (distinct same-subject NanoKVM tasks that MUST PASS — the false-positive guard). A validator that PASSes a golden-bad fixture, or FAILs golden-good / the negative-control, is itself a bluff and a release blocker.

Pre-export / pre-commit wiring

- `wire/doc_integrity_gate.sh <checkset.yaml> [repo_root]` — the HARD gate hook. Call it BEFORE any render in `sync_all_markdown_exports.sh` / Docs Chain `verify` / `commit_all.sh`; export/commit is refused on exit 1.
- `wire/CM-DOC-INTEGRITY-VALIDATION.gate.sh` — recommended pre-build gate stub (DESIGN §5.2 invariants).

Tests

```
go test -race -count=3 ./...   # unit (per family) + adapters (xlsx/sqlite/md) + runner + selfcheck
```